

Propuesta de Diseño
Instituto Tecnológico de Costa Rica
CE 3201- Taller de Diseño Digital
Laboratorio 3
Lógica Secuencial y Controladores

Estudiantes:
Juan Pablo Rodríguez Cano
Jafet Díaz Morales

26 de septiembre del 2025

A continuación, se presenta el diagrama de flujo del juego de memorización de cartas para este laboratorio.

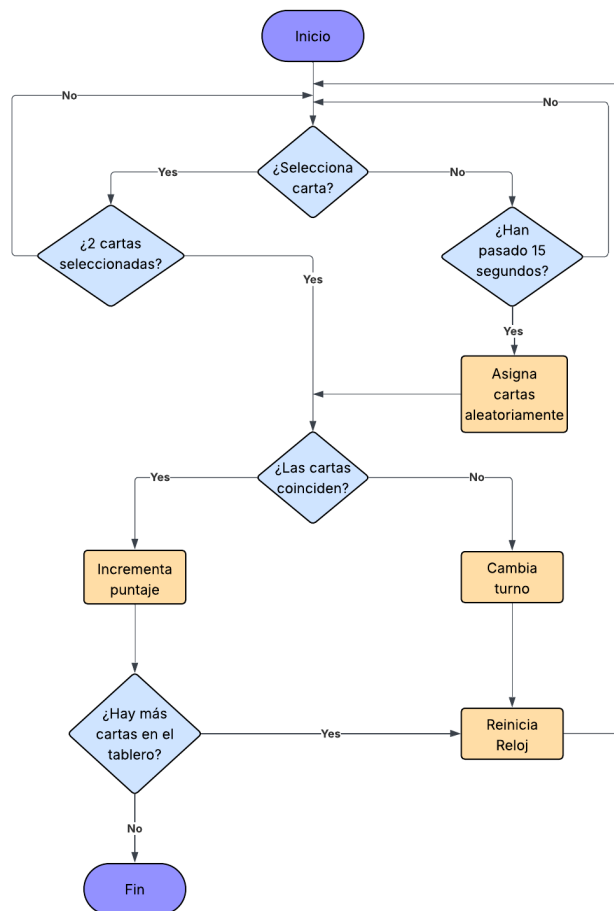


Figura 1. Diagrama de flujo de juego de memorización de cartas

De este diagrama de flujo se obtuvieron 2 diseños para la máquina de estados: una donde del estado inicial 0, se pasa al estado 1 cuando se recibe la primera selección de carta, y en este estado siguiente se espera la segunda carta. En el otro diseño, el estado cero maneja el contador de las cartas y pasa al siguiente estado hasta que reciba ambas cartas, eliminando así un estado intermedio.

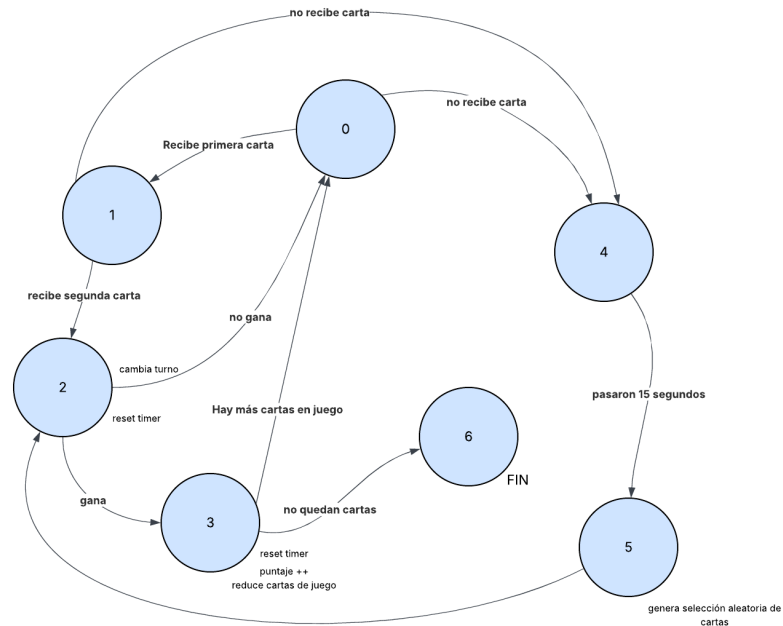


Figura 2. Diagrama de estados con estado intermedio para selección de primera carta

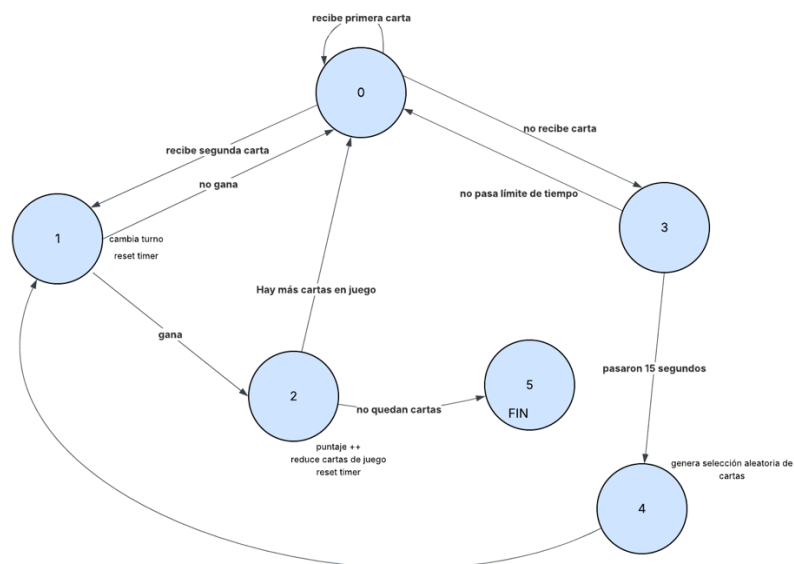


Figura 3. Diagrama de estados sin estado intermedio para número de selección de cartas

De estos diagramas se puede ver que lo único que cambia en el diseño 2, es que el estado 0 tiene más “responsabilidad” en el sentido de que en este estado se deben revisar más variables para determinar el estado siguiente. Además, a pesar de que tiene un estado menos, la cantidad de estados se debe determinar con 3 bits en ambos diseños, entonces realmente no hay una ventaja muy grande. Sin embargo, sí simplifica más el proceso y en el momento de implementar la lógica puede ayudar a tener menos fuentes de error.

A continuación, se muestran los diagramas de bloques para la implementación de la FSM con los módulos necesarios. En primera instancia, la diferencia que viene de los diseños de la máquina de estados anteriores viene de que el diseño 2 como tiene su propio estado para esperar la segunda carta, no necesita un registro contador de cartas seleccionadas ni un comparador que indique si debe cambiar de estado, si no que esto se maneja dentro del bloque. En segunda instancia, otro cambio que se considera es en los módulos necesarios para incrementar el puntaje. El primer diseño la FSM no lee el puntaje ni hace la suma, si no que la suma se hace con un módulo sumador que se agrega y la FSM solo se ocupa de poner en el sumador un 1 correspondiente. En el segundo diseño, se utiliza un registro para el puntaje del jugador seleccionado actualmente y una vez que se tiene su puntaje, la FSM lo cambia directamente. La ventaja del segundodiseño es que utiliza menos módulos, sin embargo, la complejidad de tener un bit más en el MUX que selecciona el puntaje actual y el hecho de que esto se debe hacer en 2 ciclos de reloj (leer el puntaje y actualizarlo) hace que esta implementación no sea deseada. El segundo primer agrega la complejidad de tener más módulos, pero la lógica es más simple y la FSM no debe leer el puntaje.

Se concluye que la mejor opción es el diseño 1 porque evita que el estado 0 sea más complejo, reduce sus tareas y evita la implementación de un registro contador. Además, el mecanismo para incrementar el puntaje del jugador, a pesar de que agrega un módulo más, evita la complejidad de tratar el problema secuencialmente.

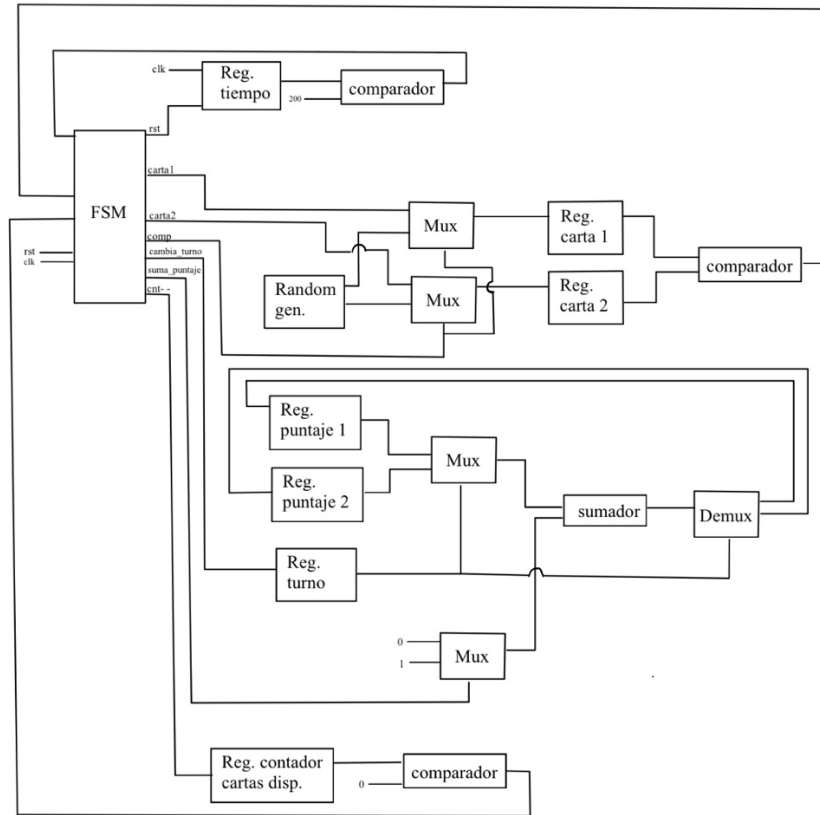


Figura 4. Diagrama de bloques de implementación del diseño 1

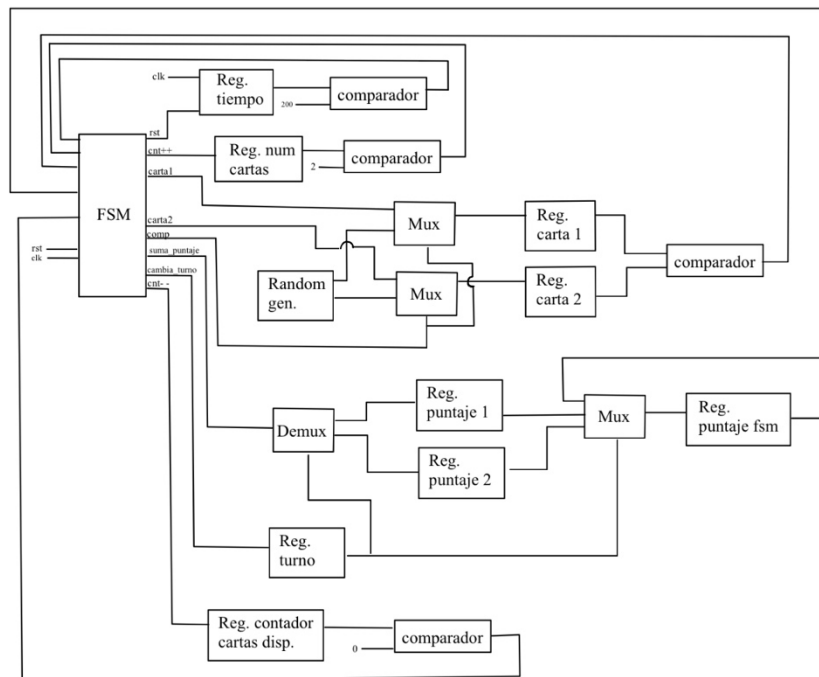


Figura 5. Diagrama de bloques de implementación del diseño 2